



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
int arr[n];
for(int i=0;i<n;i++){
scanf("%d",&arr[i]);
}
printf("Array before sorting : ");
for(int i=0;i<n;i++)
    printf("%d ",arr[i]);
printf("\n");
mergesort(arr,0,n-1);
printf("Array after Mergesort : ");
for(int i=0;i<n;i++)
    printf("%d ",arr[i]);
}
```

Program : 6(b) // C program to implement Quick Sort Algorithm

```
#include <stdio.h>
#include <time.h>
void swap(int* a, int* b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[low];
    int left = low;
    int right = high;

    while (left < right)
    {
        // Find first element greater than pivot (from starting)
        while (arr[left] <= pivot && left <= high - 1) left++;
        // Find first element smaller than pivot (from last)
        while (arr[right] > pivot && right >= low + 1) right--;

        if (left < right) swap(&arr[left], &arr[right]);
    }
    swap(&arr[low], &arr[right]);
}
```



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
        return right;
    }

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        // call partition function to find Partition Index
        int pi = partition(arr, low, high);

        // Recursively call quickSort() for left and right
        // half based on Partition Index
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int i,n,arr[20];
    clock_t t;
    double time_taken;
    t = clock();
    printf("Enter the size of array: ");
    scanf("%d",&n);
    printf("Enter the elements of array: ");
    for(i=0;i<n;i++)
        scanf("%d",&arr[i]);
    quickSort(arr, 0, n - 1);
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    t = clock()-t;
    time_taken = ((double)t)/CLOCKS_PER_SEC;
    printf("\nExecution time = %.2f milliseconds \n", time_taken);
    // getch();
    return 0;
}
```

EXPERIMENT-7

AIM:. Implement Single Source Shortest Paths using Greedy method when the graph is represented